CIS-11 Project Documentation

Team Triple A
Alicia Medrano Escobar
Aaron Pauli
Alberto Garcia
Case B (Test Calculator)
May 24, 2026

Advisor: Kasey Nguyen, PhD

Part I – Application Overview

Objectives

The main objective of this project is to create a test score calculator using LC-3 assembly language. First, the program allows the user to enter five two-digit test scores. Next, the application validates each input to ensure that it is a proper numeric value within the accepted range. After validation, the program converts each score into its corresponding letter grade and immediately displays the result to the user. Once all five scores have been entered successfully, the system calculates the maximum, minimum, and average score values.

Program Objectives

- *Prompt the user to enter five two-digit test scores*
- *Validate each input to ensure the value is within the accepted range*
- *Store valid scores into memory locations/array*
- *Convert each numerical score into a corresponding letter grade*
- *Display each score and letter grade to the user*
- *Calculate the maximum score*
- *Calculate the minimum score*
- *Calculate the average score*
- *Continuously displaying the five test scores into corresponding letter grades*
- *Display the results through the console simulator*
- *Allow the user to restart or exit out of the program*

Why are we doing this?

Furthermore, the purpose of this application is to simplify the process of score calculation by automatically processing user input and displaying accurate results efficiently. Instead of manually calculating the average, minimum, and maximum on paper with a calculator and pencil the system quickly determines the required information and presents it in a clear and organized format. The project also serves as a demonstration of the programming concepts learned throughout the course, including loops, branching, arrays, subroutines, stack operations, memory allocation, ASCII conversion, and debugging techniques within LC-3 assembly language.

Moreover, this project provides practical experience in low-level programming and strengthens problem-solving skills through the implementation of arithmetic calculations, input validation, and memory management. The intended users of the application include students, instructors, and beginner programmers who require a simple and reliable method of calculating and analyzing test score information. Instructors may also use the application as an efficient tool for processing student score data more accurately.

Business Process

Before this LC-3 program, test scores were usually calculated manually using paper, calculators, and pen. This process requires users to individually determine letter grades and manually calculate the minimum, maximum, and average values, which could take additional time and increase the possibility of errors. With this LC-3 program, the process becomes more organized and automated. The user enters five test scores, and the program checks each one to make sure it's valid before storing it. After that, it converts each score into a letter grade and shows it to the user. Once all five scores are entered, the program calculates the maximum, minimum, and average using loops and arithmetic. Finally, the user chooses whether to restart the program or end it. Overall, this makes grade calculations easier while also helping students practice important LC-3 assembly concepts.

User Roles and Responsibilities

The application is designed for individuals who need to enter and review test scores. Users are responsible for entering five valid numerical grades into the program and reviewing the calculated results displayed on the console. During program execution, the system validates each input, converts numerical scores into corresponding letter grades, and calculates the maximum, minimum, and average values automatically. This process allows users to analyze grade information more efficiently and reduces the need for manual calculations.

In addition to general users, the instructor also interacts with the system. The instructor's role is to run the program to verify that it functions correctly, confirm that the calculations and outputs are accurate, and review the documentation as part of grading the assignment. The interaction between the user and the system occurs throughout the execution process as grades are entered, validated, processed, and displayed through the console interface.

Production Rollout Considerations

To use this application, the user first downloads or opens an LC-3 simulator and loads the program file into the simulator. Once the program begins execution, the user enters five two-digit test scores through the console simulator, and the application automatically displays a corresponding letter grade for each test score while also calculating and displaying the minimum, maximum, average. Since the program only processes five test scores during each run, the storage and performance requirements are minimal.

Terminology

LC-3 which stands for little computer 3 is an educational assembly language and computer architecture used to develop and test the program. ASCII conversion refers to the process of translating keyboard character input into numerical values that the program can correctly process. A subroutine is a reusable section of code that performs a specific task and is called using JSR or JSRR before returning with RET. Branching describes the conditional instructions that control the program's flow during execution. An array is a group of consecutive memory locations used to store the five test scores entered by the user. Stack operations involve the PUSH and POP instructions, which temporarily store and retrieve data using the stack pointer. Input validation is the process of checking whether the user enters valid numerical values before any calculations are performed.

Part II – Functional Requirements

Statement of Functionality

The LC-3 application allows the user to enter five two-digit test scores through the console simulator. The program validates each input to ensure that only valid numerical values are accepted before storing the scores into the grades array in memory. After validation, the application converts each numerical score into its corresponding letter grade and displays the result of the average, minimum, and maximum score to the user using ASCII conversion operations.

Furthermore, the program calculates the maximum score, minimum score, and average score using arithmetic operations, arrays, loops, and branching instructions. The application also includes multiple subroutines for tasks such as input validation, grade conversion, stack operations, integer formatting, and register clearing. PUSH and POP operations are implemented to temporarily store and retrieve letter grades during execution. Additionally, the program includes restart functionality that allows the user to rerun the application without restarting the LC-3 simulator manually.

Scope

The scope of this project includes creating and testing a test score calculator using LC-3 assembly language. The program is limited to processing five test scores and focuses on concepts learned throughout the course, such as loops, branching, arrays, subroutines, stack operations, and input validation. Features like GPA calculations or displaying a pass or fail text next to the five test scores are not included in the current version of the program.

Performance

The program is designed to process and display user entered test score information efficiently without any noticeable delay during normal execution conditions. After the user enters a two-digit score, the application validates the input, determines the corresponding letter grade, and displays the result in less than one second under standard execution conditions. Additionally, the system stores and processes five test scores during each execution cycle. The calculations for the maximum grade, minimum grade, and average grade are completed and displayed within approximately two seconds after the fifth score has been entered.

Furthermore, the program correctly restarts and clears register values when the user chooses to run the application again, allowing the user to begin with a clean execution state. If an invalid input outside the accepted range of 0–99 is entered, the application immediately displays an error message and restarts the program in less than one second. Finally, the system is designed to maintain accurate test score calculations and letter grade conversions for all valid numerical inputs entered by the user.

Usability

The program will provide a simple text-based console program that allows users to enter grades with very minimal instruction. All prompts and outputs will be displayed clearly using direct wording, such as "Enter 5 scores (0-99)" and labeled outputs for the minimum, maximum, and average values. Additionally, users will be able to complete one full execution of the program, including entering five grades and viewing the results, in under about 2 minutes without extra help. Furthermore, the program will provide immediate results after each score is entered by displaying the corresponding letter grade. If an invalid entry is detected, the system will display the message "INVALID ENTRY, RESTARTING..." so the user understands why the program restarted. Finally, the program will require no more than two inputs per grade entry and will allow users to restart the program through a simple Y/N option at the end of execution, improving usability and efficiency.

Documenting Requests for Enhancements

The following table lists potential future enhancements that could be added in later versions of the application.

Date	Enhancement	Requested by	Notes	Priority	Release No/ Status
5/12/26	Displaying the equivalent letter grade with the results	Alicia M.	-Max, Min, and Avg will also have a letter grade side by side.	No	No Status
5/13/26	GPA added onto the program	Aaron P.	-Converting the Avg into a GPA scale	No	No Status
5/16/26	(0-100) scale added onto the program	Alberto G.	-Adding a scale of 0 to 100 next to the letter grade when shown	No	No Status
5/20/26	Add a Fail or Pass remark next to the scores that correlates it	Alberto G	-Fail and Pass message will be displayed next to the scores/grades that correlate with it.	No	No Status

Part III – Appendices

Grade Conversion Scale

Numeric Score	Letter Grade
90 – 100	A
80 – 89	B
70 – 79	C
60 – 69	D
0 – 59	F

Example Scenarios

Scenario 1:

User Input:
95
88
76
64
91

Program Output:
MAX 95 A
MIN 64 D
AVG 82 B

Scenario 2:

User Input:
50
60
70
80
90

Program Output:
MAX 90 A
MIN 50 F
AVG 70 C

Developer Information

Main Subroutines	Purpose
GET_GRADE	Reads and stores the grade input
GET_LETTER	Converts to a numeric letter grade
VALIDA	Validates the numeric input
BREAK_INT	Splits the integer digits for the output
PUSH/POP	Works to stack operations
CLEAR_REG	Clears the register values

Registers	Purpose
R0	Input/Output Register & ASCII character handling
R1	Manages counters, comparisons, division logic, & loop control
R2	Manages array pointers, temporary arithmetic values, and validation calculations
R3	Stores main grades, totals, and average calculations
R4	A temporary arithmetic and comparison register
R5	A temporary storage for comparisons and integer formatting
R6	Stack pointer for counters and the temporary storage
R7	The return address register for subroutines (JSR / RET)

Pseudo-Code

- *Subroutines included:*
- *The pseudocode defines and labels all major routines (LOOP_INPUT, GET_GRADE, CHECK_MAX, CHECK_MIN, CALC_AVG, GET_LETTER, END_PROGRAM), satisfying the rubric requirement for subroutines.*

- *Branching demonstrated:*
- *Conditional decisions appear throughout the pseudocode (e.g., IF score ≥ 90, IF current grade > highest grade, IF user enters Y), fulfilling the branching requirement.*

- *Conditional tasks present:*
- *Multiple ELSE IF and ELSE structures are used to handle different grade ranges and program-restart logic, meeting the conditional-logic requirement.*

- *Iterative tasks included:*
- *The pseudocode uses WHILE loops and FOR loops (e.g., looping through 5 scores, iterating through stored grades), satisfying the iterative-task requirement.*

- *Labels used for clarity:*
- *Each major section is labeled (e.g., LOOP_INPUT, GET_GRADE, CHECK_MAX), meeting the requirement for labeled sections.*

```
START  Program

DISPLAY the welcome message

DISPLAY "Enter 5 test scores (0 - 99)"

CALL LOOP_INPUT

CALL CHECK_MAX

CALL CHECK_MIN

CALL CALC_AVG

CALL END_PROGRAM

LOOP_INPUT

SET the score counter = 0

WHILE the number of scores is less than 5

        CALL GET_GRADE

        STORE score in the GRADES array

        CALL GET_LETTER
```

```
        PRINT the letter grade
        INCREMENT the counter by one
END WHILE
RETURN

GET_GRADE
INPUT first digit
VERIFY the digit is valid
INPUT the second digit
VERIFY digit is valid
COMBINE digits into one score
VERIFY grade
RETURN the grade to LOOP_INPUT
CHECK_MAX
SET the highest grade = to the first stored grade
FOR each remaining grade
        IF current grade > highest grade
        SET highest grade = current grade
        END IF
END FOR
DISPLAY "MAX" as the max score
PRINT highest grade by CHECK_MAX
CALL GET_LETTER
PRINT the corresponding letter grade
RETURN

CHECK_MIN

SET the lowest grade = to the first stored grade
FOR each remaining grade
        IF current grade < lowest grade
        SET lowest grade = current grade
        END IF
```

```
END FOR
DISPLAY "MIN" as the minimum score
PRINT lowest score by CHECK_MIN

CALL GET_LETTER
PRINT the corresponding letter grade
RETURN
CALC_AVG
SET the total = 0
FOR each stored grade
        ADD current grade to total
END FOR
DIVIDE total by 5
STORE the result as the average overall grade
DISPLAY "AVG"
PRINT average score

CALL GET_LETTER
PRINT the corresponding letter grade
RETURN


GET_LETTER
IF the score >= 90
        ASSIGN the letter grade of A
ELSE IF score >= 80
        ASSIGN the letter grade B
ELSE IF the score >= 70
        ASSIGN the letter grade C
ELSE IF the score >= 60
        ASSIGN the letter grade D
ELSE
        ASSIGN the letter grade F
END IF
RETURN the appropriate corresponding letter grade
```

```
END_PROGRAM

DISPLAY "Program Finished"

DISPLAY "Run program again? Y/N"

INPUT user choice

IF user enters Y

        RESTART the program anew

ELSE

        END the program

END IF

RETURN
```